



Comunicação Interprocessos

— Aula 4

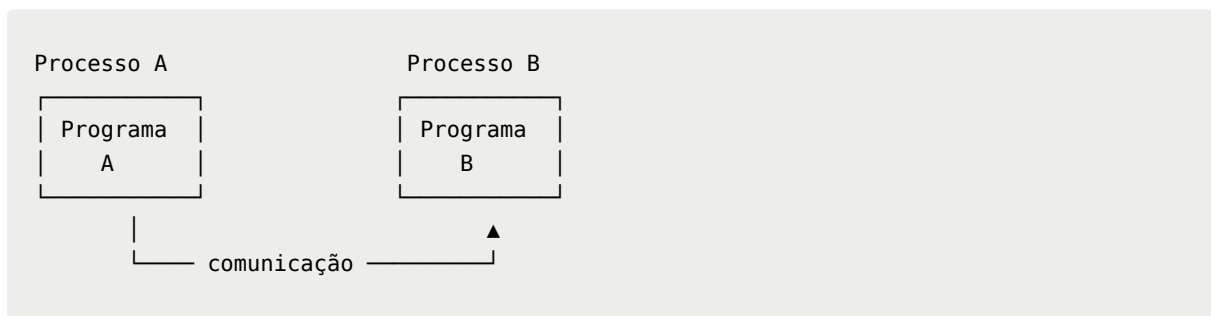
🕒 Criado em	7 de setembro de 2026 09:54
🏷️ Tags	

1. O que é Comunicação Interprocessos?

IPC (Inter-Process Communication) é o conjunto de mecanismos que permite que **processos troquem informações e coordenem suas ações**.

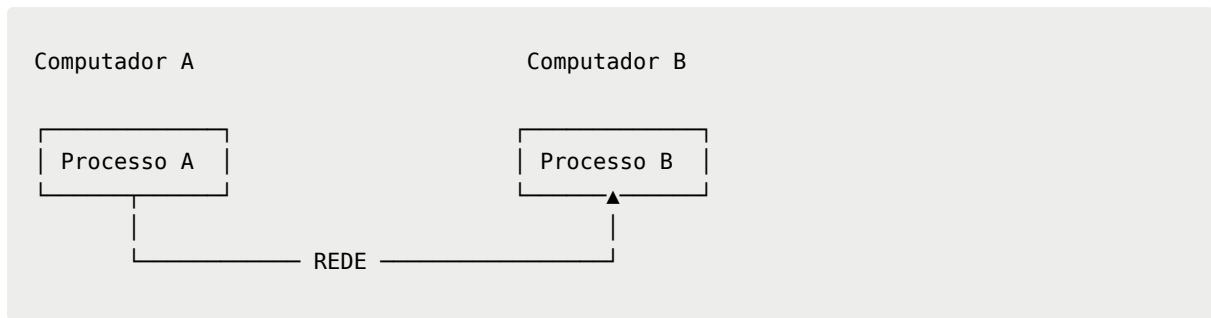
Um processo é, de forma simplificada, um programa em execução.

Por exemplo:



Em um mesmo computador, processos podem se comunicar usando mecanismos do próprio sistema operacional.

Em **Sistemas Distribuídos**, porém, existe uma situação mais interessante:



Agora os processos estão em máquinas diferentes.

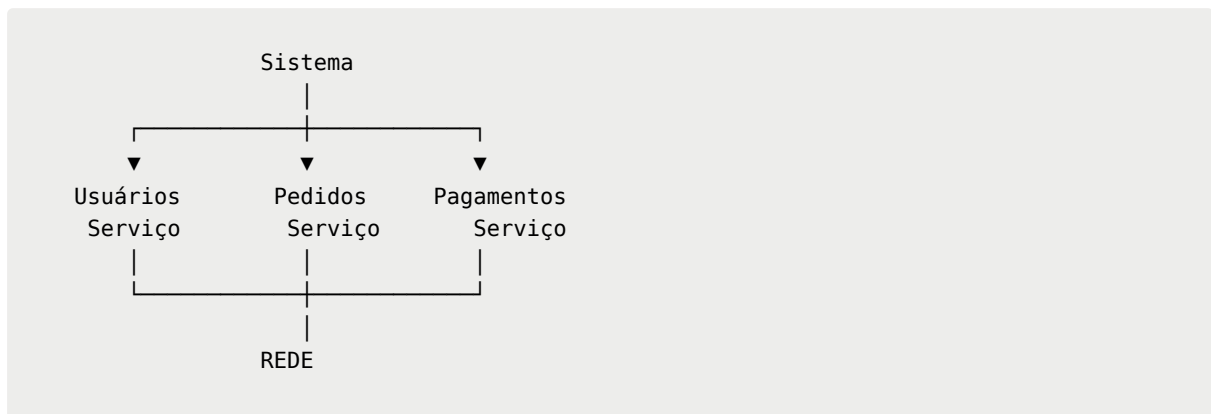
Eles precisam utilizar uma **rede** para trocar informações.

É aí que entram **TCP, UDP e sockets**.

2. Por que processos precisam se comunicar?

Porque um sistema distribuído normalmente divide responsabilidades entre diferentes componentes.

Imagine uma aplicação de e-commerce:



O serviço de pedidos pode precisar perguntar ao serviço de usuários:

| "Esse usuário existe?"

E depois perguntar ao pagamento:

| "O pagamento foi aprovado?"

Esses componentes precisam trocar mensagens.

Portanto:

A comunicação é um dos elementos fundamentais de um sistema distribuído.

Sem comunicação, os componentes seriam apenas programas independentes.

3. Princípios de rede aplicados à comunicação

Para entender IPC distribuído, precisamos lembrar alguns conceitos básicos de redes.

Uma comunicação normalmente envolve:



Para que isso funcione, precisamos identificar **quem está enviando, para quem os dados devem ir e como eles serão transportados**.

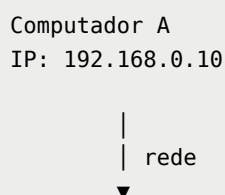
4. Endereço IP

O **IP** identifica uma interface de rede dentro de uma rede.

Por exemplo:

192.168.0.10

Podemos imaginar:



```
Computador B  
IP: 192.168.0.20
```

O IP ajuda a determinar **para qual máquina os dados devem ser enviados**.

Mas existe um problema.

Uma mesma máquina pode executar vários processos.

Por exemplo:

```
Computador  
IP: 192.168.0.10  
  
├─ navegador  
├─ servidor web  
├─ banco de dados  
└─ outro serviço
```

Como saber qual processo deve receber os dados?

É aí que entra a **porta**.

5. Portas

Uma **porta** identifica um ponto de comunicação associado a um processo/serviço em um host.

Podemos pensar:

```
IP + Porta
```

como algo semelhante a:

```
endereço do prédio + número do apartamento
```

Por exemplo:

```
192.168.0.10:8080
```

Aqui temos:

```
192.168.0.10 → máquina  
8080         → porta
```

Assim, uma aplicação pode estar escutando:

```
192.168.0.10:8080
```

enquanto outra utiliza:

```
192.168.0.10:3306
```

6. Socket

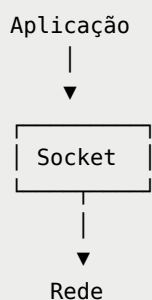
Agora chegamos ao conceito principal.

Um **socket** é uma interface que permite que uma aplicação se comunique através da rede.

Uma forma simplificada de pensar é:

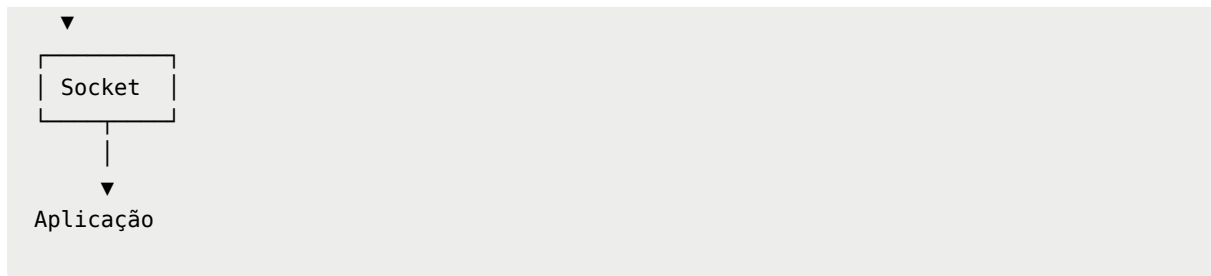
█ **Socket = ponto de comunicação entre uma aplicação e a rede.**

Imagine:

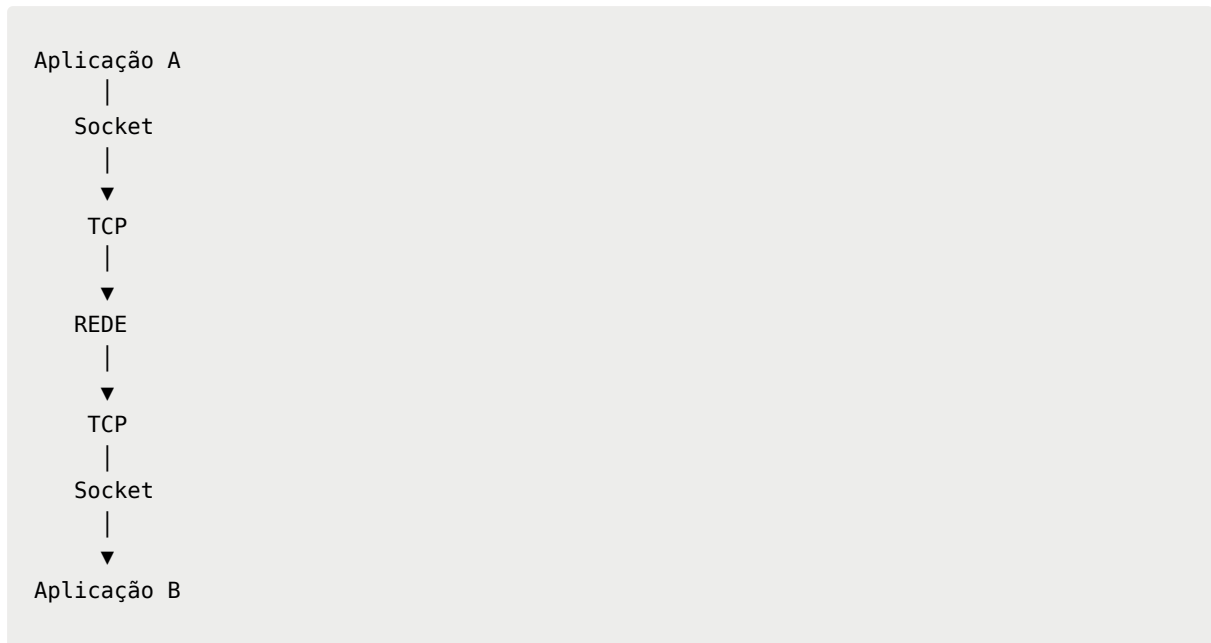


E no outro computador:

```
Rede  
|
```



Então podemos ter:



O socket não é exatamente o TCP ou o UDP.

Ele é a **interface usada pela aplicação para utilizar mecanismos de comunicação da rede**.

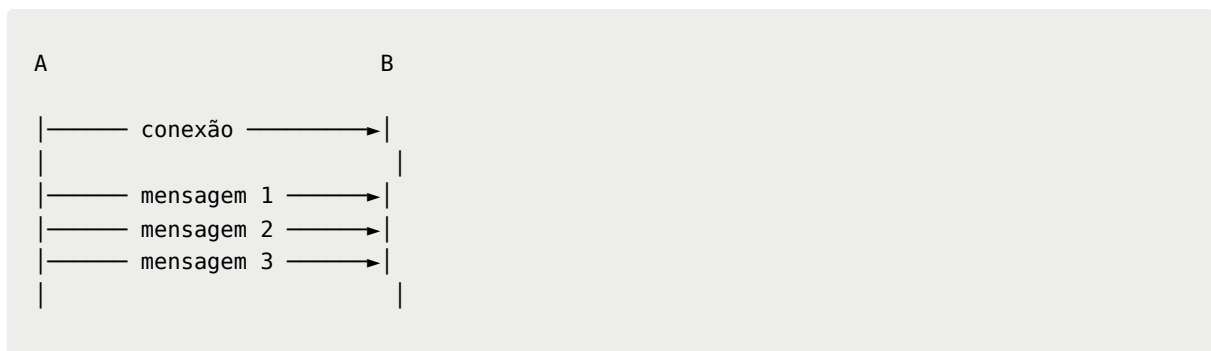
7. TCP

TCP (Transmission Control Protocol) é um protocolo de transporte orientado à conexão.

A ideia principal é fornecer uma comunicação:

- confiável;
- ordenada;
- baseada em conexão;
- com mecanismos para detectar perda e retransmitir dados.

Imagine:



O TCP trabalha para que os dados sejam recebidos de maneira confiável e na ordem correta.

Exemplo conceitual

Se enviamos:

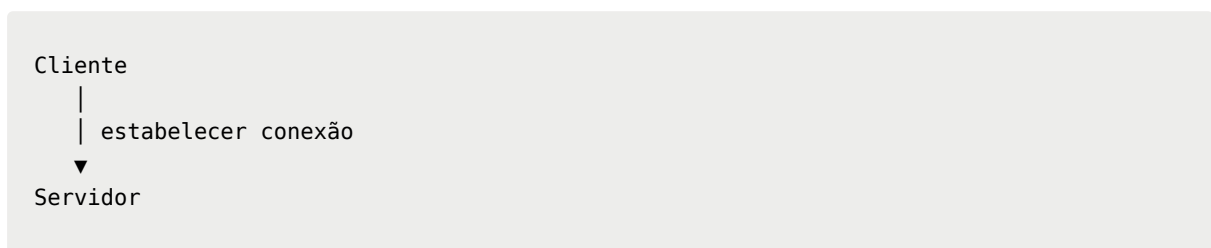
A B C D

A aplicação que recebe os dados espera obter a sequência correspondente, e o TCP possui mecanismos para lidar com perdas e reordenamento.

8. Características importantes do TCP

Conexão

Antes da troca normal de dados, existe um processo de estabelecimento de conexão.



Confiabilidade

O TCP possui mecanismos para detectar perdas e retransmitir dados.

Ordenação

Os dados são organizados para que a aplicação receba o fluxo na ordem apropriada.

Controle de fluxo e congestionamento

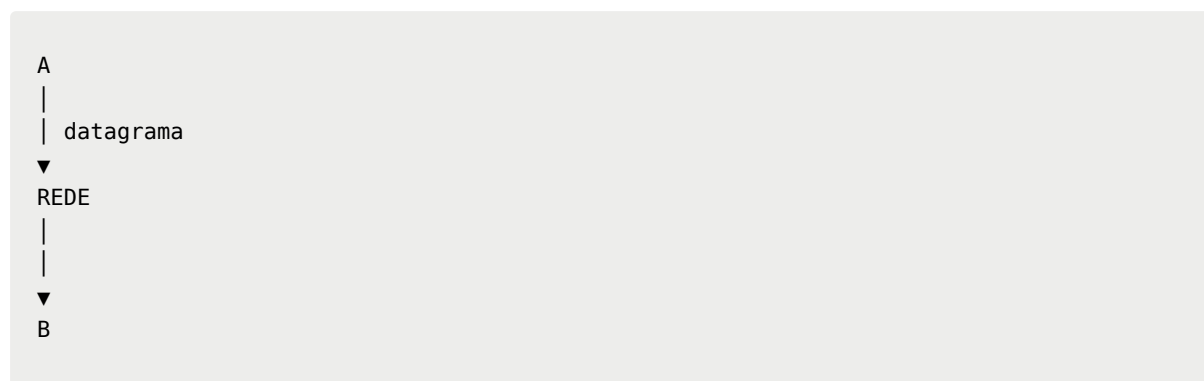
O protocolo possui mecanismos para evitar que um receptor seja inundado e para adaptar a transmissão às condições da rede.

9. UDP

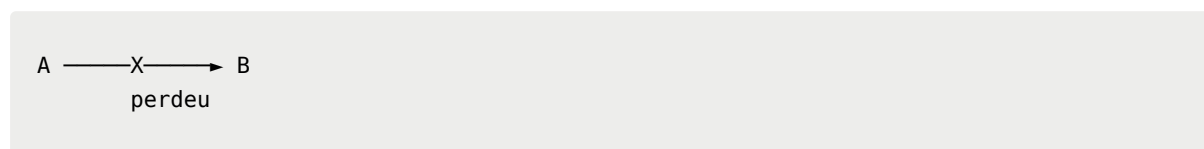
UDP (User Datagram Protocol) é um protocolo de transporte mais simples.

Diferentemente do TCP, ele não estabelece uma conexão da mesma maneira e **não oferece garantia de entrega, ordenação ou retransmissão pela própria camada UDP.**

A ideia pode ser representada assim:



Se o datagrama for perdido:



O UDP não vai automaticamente retransmiti-lo.

10. TCP × UDP

Essa comparação é importante para prova e também para o mercado.

Característica	TCP	UDP
Conexão	Orientado à conexão	Sem conexão

Característica	TCP	UDP
Entrega confiável	Sim, pelo protocolo	Não
Ordenação	Sim	Não
Retransmissão	Sim	Não
Controle de congestionamento	Sim	Não da mesma forma que TCP
Sobrecarga	Maior	Menor
Latência	Pode ser maior	Pode ser menor
Modelo	Fluxo de bytes	Datagramas

Uma observação importante

Não pense:

TCP = bom / UDP = ruim

ou:

TCP = lento / UDP = rápido

Isso é simplificação demais.

A escolha depende do problema.

11. Quando TCP faz sentido?

TCP é interessante quando **perder ou receber dados fora de ordem é problemático**.

Exemplos:

- APIs HTTP/HTTPS tradicionais;
- transferência de arquivos;
- comunicação com bancos de dados;
- sistemas onde a confiabilidade é essencial.

Imagine uma transferência:

Arquivo:
[Parte 1][Parte 2][Parte 3][Parte 4]

Perder uma parte pode comprometer o resultado.

Por isso, mecanismos de confiabilidade são importantes.

12. Quando UDP pode fazer sentido?

UDP pode ser interessante quando a aplicação prioriza:

- baixa latência;
- baixa sobrecarga;
- comunicação em tempo real;
- tolerância a alguma perda;
- controle feito pela própria aplicação ou por protocolos construídos sobre UDP.

Exemplos de contextos:

- jogos online;
- streaming em tempo real;
- comunicação multimídia;
- descoberta de serviços;
- DNS tradicional.

A ideia não é simplesmente:

┆ "UDP é mais rápido."

É:

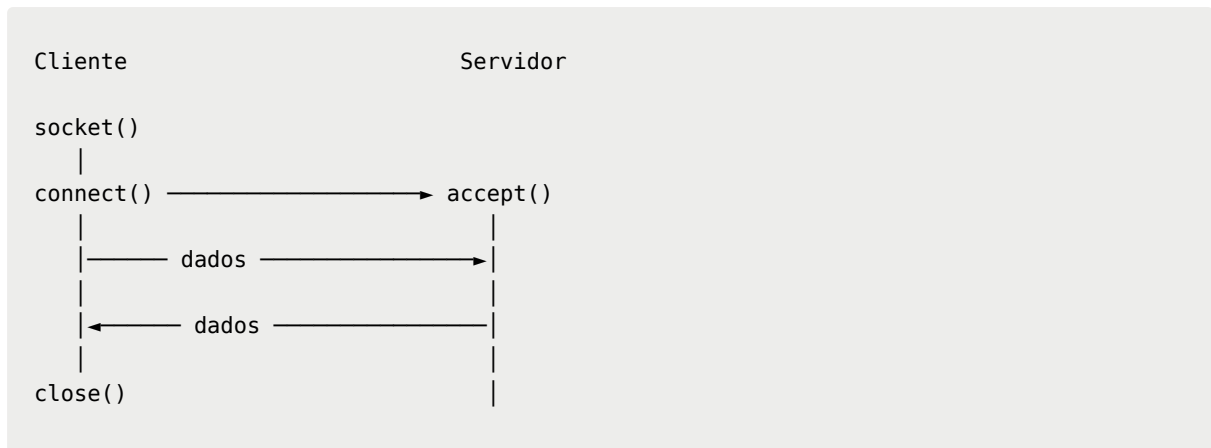
┆ **UDP oferece menos garantias e menos mecanismos no transporte, o que pode ser útil quando a aplicação prefere lidar com essas questões por conta própria ou quando determinadas perdas são aceitáveis.**

13. Socket TCP × Socket UDP

Quando programamos com sockets, a forma de comunicação muda de acordo com o protocolo.

TCP

Temos uma comunicação baseada em conexão:

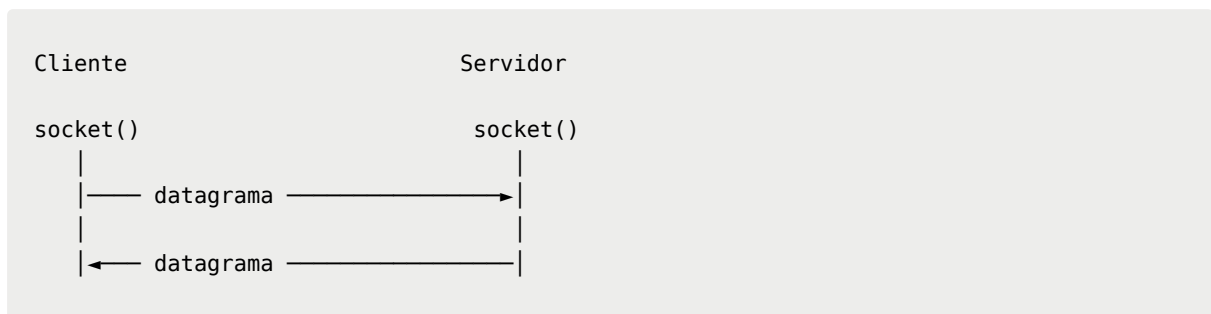


O servidor normalmente fica esperando conexões.

UDP

Não precisamos estabelecer uma conexão TCP.

A comunicação pode ser pensada como envio de datagramas:



A aplicação trabalha diretamente com mensagens/datagramas.

14. Exemplo conceitual com Java

Como você está estudando Java, isso é especialmente interessante.

Um servidor TCP pode ser construído conceitualmente usando:

```
ServerSocket
```

e o cliente usando:

Socket

A estrutura seria aproximadamente:

```
Servidor
|
ServerSocket
|
accept()
|
Socket
|
▼
Comunicação
```

Enquanto o cliente:

```
Cliente
|
Socket
|
connecta ao servidor
|
▼
Comunicação
```

Para UDP, Java possui classes como:

```
DatagramSocket
DatagramPacket
```

Nesse caso, trabalhamos com **datagramas**, em vez de um fluxo TCP.

Isso é uma das partes em que a matéria pode deixar de ser apenas teórica e começar a virar programação de verdade.

15. O que acontece por trás de uma API?

Aqui está uma conexão MUITO importante com o mercado.

Quando você faz:

```
Angular
|
| GET /livros
▼
Spring Boot
```

Você provavelmente pensa:

|"Estou fazendo uma requisição HTTP."

E está correto.

Mas existe uma pilha de comunicação por trás:

```
Angular
|
| HTTP
▼
TCP
|
▼
IP
|
▼
Rede
|
▼
IP
|
▼
TCP
|
▼
HTTP
|
▼
Spring Boot
```

Simplificando bastante, temos:

```
Aplicação
↓
HTTP
↓
TCP
↓
IP
```

↓
Rede

É por isso que aprender **TCP, UDP e sockets** não é uma coisa completamente separada do desenvolvimento web.

Você está estudando uma parte da infraestrutura que fica **abaixo das APIs que você já utiliza**.

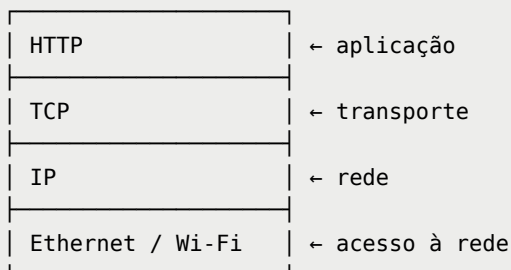
16. E onde entra HTTP?

Essa distinção é importante:

HTTP não é TCP.

São protocolos de camadas diferentes.

Uma simplificação da pilha seria:



Quando você utiliza uma API REST:

```
GET /livros
```

isso é **HTTP**.

O HTTP tradicionalmente utiliza **TCP** como transporte, enquanto versões mais modernas do HTTP também podem utilizar outros mecanismos, como o HTTP/3 sobre QUIC, que roda sobre UDP.

Isso é uma conexão futura muito interessante: você vai perceber que protocolos de aplicação podem evoluir sem abandonar os conceitos fundamentais de comunicação em rede.

17. O grande problema: a rede não é perfeita

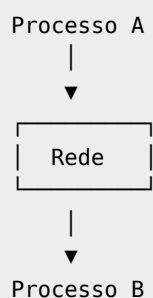
Esse talvez seja o ponto mais importante para Sistemas Distribuídos.

Quando dois processos estão na mesma máquina:

```
Processo A → Processo B
```

a comunicação pode ser relativamente simples.

Mas quando estão separados:



podem acontecer:

```
mensagem perdida  
mensagem atrasada  
mensagem duplicada  
conexão interrompida  
servidor indisponível  
rede congestionada
```

Portanto, quando você programa sistemas distribuídos, precisa pensar:

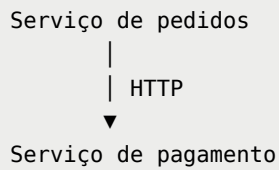
| "E se a comunicação não acontecer como eu espero?"

Essa pergunta vai aparecer repetidamente ao longo da disciplina.

18. Ligação com o mercado

Imagine que você esteja trabalhando em um backend Spring Boot.

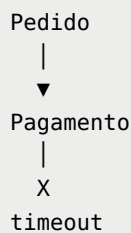
Você possui:



Você não pode programar assumindo:

|"O pagamento sempre vai responder."

Pode acontecer:



Então você precisa pensar em:

- timeout;
- retry;
- idempotência;
- circuit breaker;
- tratamento de erros;
- observabilidade;
- mensagens assíncronas.

E aqui começa uma conexão muito interessante:

Sockets/TCP/UDP são os fundamentos.

Depois você aprende:

HTTP → APIs → RPC → mensageria → microsserviços → sistemas distribuídos em produção.



Resumo para guardar

Comunicação Interprocessos

| Permite que processos troquem informações e coordenem suas atividades.

Socket

| Interface utilizada por uma aplicação para realizar comunicação através da rede.

IP

| Identifica o destino na rede.

Porta

| Permite identificar um ponto de comunicação/serviço em um host.

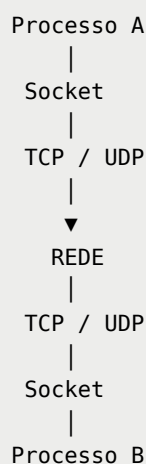
TCP

| Transporte orientado à conexão, com mecanismos de confiabilidade e ordenação.

UDP

| Transporte baseado em datagramas, sem garantia de entrega ou ordenação pelo próprio protocolo.

A grande ideia





Perguntas para estudo ativo

Depois de estudar, tente responder sem consultar o material:

1. O que significa IPC?
2. Por que comunicação entre processos é fundamental em sistemas distribuídos?
3. Qual é a função de um socket?
4. Qual é a diferença entre IP e porta?
5. TCP e UDP pertencem a qual camada?
6. Qual é a principal diferença entre TCP e UDP?
7. Por que TCP é chamado de orientado à conexão?
8. O que acontece se um datagrama UDP for perdido?
9. Por que UDP pode ser útil em aplicações de tempo real?
10. HTTP e TCP são a mesma coisa? Explique.
11. O que existe, de maneira simplificada, entre uma aplicação e outra quando elas se comunicam pela rede?
12. Por que uma API Spring Boot pode ser considerada parte de uma comunicação distribuída?
13. O que pode acontecer quando uma mensagem enviada por um serviço não recebe resposta?
14. Por que um desenvolvedor de backend precisa se preocupar com problemas de rede mesmo usando abstrações como HTTP?



Desafio

Imagine que você tenha dois programas Java:

Programa A

e

Programa B

O Programa A precisa enviar uma mensagem para o Programa B através da rede.

Tente desenhar **do zero** o caminho dessa mensagem, começando em:

Programa A

e terminando em:

Programa B

incluindo **socket, TCP/UDP, IP e rede**.

Se você conseguir desenhar e explicar esse caminho com suas próprias palavras, você já entendeu o núcleo dessa parte da Unidade I.